

Pipeline Hazards in Computer Architecture

Pipeline hazards are situations that prevent the next instruction in the pipeline from executing during its designated clock cycle, causing stalls or bubbles. Hazards reduce the efficiency of pipelining, which ideally aims for one instruction per clock cycle.

Types of Pipeline Hazards

There are three main types of hazards in pipelined processors:

1. Structural Hazards

Structural hazards occur when hardware resources are insufficient to support all overlapped instructions concurrently. For example, if two instructions require the same memory or functional unit simultaneously, one must wait, causing a stall. Proper hardware design, such as duplicating resources, is used to avoid these hazards.

2. Data Hazards

Data hazards happen when an instruction depends on data from a previous instruction that has not yet completed. This dependency requires the pipeline to stall or use techniques to resolve the hazard.

-

Types of Data Hazards:

-

Read After Write (RAW): Occurs when an instruction tries to read a location before a previous instruction writes to it.

-

Write After Read (WAR): Happens when an instruction writes to a location before a previous instruction reads it.

-

Write After Write (WAW): Arises when two instructions write to the same location in an incorrect order.

-

Handling Data Hazards:

-

Operand Forwarding: Bypassing data directly from one pipeline stage to another instead of waiting for register write-back.

-

Instruction Reordering: Reordering instructions in software to avoid hazards.

-

Stalling: Inserting bubbles or no-operation cycles until the hazard clears.

3. Control Hazards

Control hazards (branch hazards) arise due to branching instructions that change the flow of instruction execution. The pipeline may fetch incorrect instructions before the branch decision is known, leading to wasted cycles when flushing those instructions.

-

Handling Control Hazards:

-

Branch Prediction: Predicting whether a branch will be taken and fetching accordingly.

-

Delayed Branching: Scheduling useful instructions in the delay slot after the branch.

-

Branch Target Buffers and Dynamic Prediction: Using history-based prediction to improve accuracy and reduce stall penalties.

Advantages of Pipelining

-

Increased Instruction Throughput: Multiple instructions are processed simultaneously in different stages, improving overall performance.

- **Better Utilization of Hardware:** Efficient use of CPU functional units by overlapping instruction execution.
 - **Higher Clock Speed Potential:** Breaking instruction execution into smaller steps allows higher frequency operation.
 - **Improved Performance on Repetitive Tasks:** Streaming data and repeated operations are handled effectively.
-

Disadvantages of Pipelining

- **Increased Complexity:** Requires complex hazard detection, forwarding, and control logic.
- **Pipeline Stalls:** Hazards cause bubbles or idle cycles that reduce effective throughput.
- **Difficult Handling of Branches:** Control hazards from branching increase pipeline penalties.
- **Increased Power and Area:** Additional hardware for forwarding and hazard detection consumes more power and chip area.
- **Latency of Individual Instructions:** Overall instruction latency may increase despite higher throughput.